AI PROJECT SHOWCASE | EDGE AI | NO CLOUD

FXR90 Edge Maintenance Chatbot

A no-cloud maintenance assistant that runs on the reader as a user application, answers local diagnostic questions, and explains reader, antenna, and cable health using on-device telemetry.

On-Device App | No Cloud | Local Telemetry | Maintenance Assistant

DEB App
Installable FXR90 user application

Offline
Rules, local retrieval, and telemetry context

Private
Reader data stays on the device or local network

PROJECT SUMMARY

Yes, a no-cloud chatbot is possible on the device

The FXR90 API supports custom user applications, so a local maintenance chatbot can be packaged as a reader app and run without sending telemetry to the cloud. The practical first version should be a rule-based and retrieval-based chatbot that explains reader health, antenna degradation, cable risk, errors, warnings, and recommended actions from local data.

Local
Runs as an installed user app on the reader.

Explainable
Uses known rules, scores, evidence, and SOP content.

Useful
Answers real maintenance questions using live device data.

Lightweight
Avoids heavy cloud LLM dependency for MVP.

What The Chatbot Can Answer

- Why is Antenna 3 high risk?
- Which antenna should I inspect first?
- Is the issue cable loss, antenna misalignment, or disconnect?
- What changed in the last 24 hours?
- What action should the technician take now?

What It Should Not Try First

- A full ChatGPT-style model running entirely on the reader.
- Large language models that require high RAM, GPU, or cloud calls.
- Open-ended internet knowledge that cannot be verified locally.
- Unapproved automatic configuration changes without operator review.

Recommended MVP

Build an **Edge Maintenance Chatbot v1** using local rules, local retrieval, and the antenna/reader health scoring engine. It can feel conversational while staying small, explainable, private, and reliable on the reader.

Source capabilities reviewed from docs/openapi_md.json: install_user_app, start_user_app, stop_user_app, autostart_user_app, get_user_apps, set_req_usr_app, userapp_event, and heartbeat userapps telemetry.

USER APP SUPPORT

The reader already has the hooks needed for an edge chatbot

The API does not require the chatbot to live in the cloud. A custom app can be installed, started, monitored, sent requests, and emit responses through the reader's management event path.

API Capability	Relevant Command/Event	How It Helps The Chatbot
Install App	`install_user_app`	Deploy the chatbot as a Zebra-approved `.deb` package from an HTTP(S) file server.
Run App	`start_user_app` `stop_user_app`	Start, restart, or stop the chatbot during deployment and maintenance.
Boot Behavior	`autostart_user_app`	Make the chatbot automatically start when the reader boots.
App Inventory	`get_user_apps`	Confirm the chatbot is installed, running, and configured for autostart.
Ask Question	`set_req_usr_app`	Send a structured prompt or command to the chatbot app running on the reader.
Return Answer	`userapp_event`	Emit the answer, evidence, and recommended action as a custom user-app event.
Monitor App	`heartbeat.userapps[]`	Track chatbot CPU, RAM, uptime, running status, and buffer health.

Important Constraint

The API proves that custom user apps are supported. It does not guarantee enough hardware resources for a large local LLM. The safe design is to start with rules and local retrieval, then test whether a tiny quantized model is practical on the actual device.

SYSTEM DESIGN

Keep the assistant, data, and reasoning on the edge

The chatbot should use local telemetry and a compact knowledge base. It can expose a local HTTP page, accept `set_req_usr_app` requests, or both. No cloud request is needed for normal operation.

1. Local Data

Heartbeat, status snapshots, tag reads, antenna scores, warnings, errors, and local SOP documents.

heartbeat · tag events · scores · SOPs

->

2. Chatbot App

Intent parser, retrieval engine, scoring tools, answer templates, local database, and safety guardrails.

rules · retrieval · SQLite · guardrails

->

3. Local Answer

Returns explanation, evidence, risk level, and recommended maintenance action without cloud transfer.

local UI · userapp_event · JSON

```
Question Flow
1. Receive question
2. Detect intent and entities
3. Pull local telemetry and score evidence
4. Retrieve matching maintenance SOP
5. Generate structured answer
6. Emit answer locally
```

Example Interfaces

- **Reader command path:** send prompt through `set_req_usr_app` and return via `userapp_event`.
- **Local web page:** small technician UI hosted by the app on the reader LAN interface.
- **Local API:** REST endpoint for a nearby HMI, handheld, or maintenance tablet.
- **Optional local gateway:** use a site server for larger LLMs while still avoiding cloud.

IMPLEMENTATION CHOICE

Use the right level of AI for the reader hardware

There are three realistic designs. The best first version is the smallest and most reliable: rules plus retrieval. A tiny local model or local LAN-based model can be added later after hardware testing.

Option	Where It Runs	Pros	Risks	Recommendation
Rules + Retrieval	On FXR90 user app	Small, explainable, fast, private, easy to package.	Less flexible for broad open-ended questions.	**MVP choice**
Tiny Local LLM	On FXR90 if resources allow	More natural language flexibility while staying offline.	RAM, CPU, storage, latency, heat, and package size constraints.	Lab test only
Local Edge Gateway LLM	Industrial PC or local server on same LAN	Much stronger model without cloud data transfer.	Requires another device and local network dependency.	Best for v2

Rule Engine

- Maps known intents to local tools.
- Uses thresholds and scoring evidence.
- Produces consistent technician answers.

Retrieval Layer

- Looks up local SOPs and troubleshooting notes.
- Returns relevant steps for antenna, cable, network, and thermal issues.
- Works fully offline.

Answer Builder

- Combines intent, telemetry, and SOP content.
- Always includes evidence and recommended action.
- Avoids unsupported claims.

Design Principle

Treat the chatbot as a maintenance decision assistant, not a generic internet chatbot. Its job is to explain local reader state and guide safe field action.

USER EXPERIENCE

Technicians ask simple questions and get evidence-based answers

The chatbot should be direct, operational, and auditable. Every answer should explain the signal behind the recommendation.

FXR90 Edge Maintenance Assistant

Reader: FXR90-01 | Mode: Offline | Data source: Local telemetry

No Cloud

Why is Antenna 3 high risk?

Antenna 3 is high risk because read volume is down 42% from the 14-day baseline and median RSSI is 6.8 dB weaker. The antenna is still connected, and mode/transmit power did not change. Likely cause: cable loss, loose connector, or antenna misalignment. Recommended action: inspect cable path, tighten connectors, check antenna angle, then run a validation scan.

Is this a reader issue or only antenna issue?

Current evidence points to an antenna-path issue. Other antennas are within baseline, reader CPU/RAM are normal, radio connection is active, and there are no current reader temperature warnings.

Supported Question Types

- Current reader health and risk.
- Antenna degradation explanation.
- Cable compensation and read-point checks.
- Warnings and errors summary.
- Recommended technician action.
- Recent changes in the last hour, shift, or day.

Answer Format

- **Short answer:** direct diagnosis.
- **Evidence:** readings, trend, event, or score.
- **Likely cause:** ranked issue type.
- **Action:** safe next step.
- **Confidence:** based on signal completeness.

DELIVERY PLAN

Build an on-device assistant in controlled phases

The first release should be small: local telemetry cache, health scoring, intent matching, troubleshooting retrieval, and structured answers. The system can later add a tiny offline model or local gateway integration.

1
Package
Create a minimal Linux service and package it as an installable `.deb`.

2
Collect
Cache heartbeat, status, antenna scores, tag-read summaries, warnings, and errors locally.

3
Answer
Implement intents, evidence lookup, SOP retrieval, and answer templates.

4
Expose
Support `set_req_usr_app`, `userapp_event`, and optional local web UI.

5
Harden
Add resource limits, logs, auth, app monitoring, and safe fallback behavior.

MVP Scope

- Installable chatbot user app.
- Local SQLite telemetry cache.
- Intent matching for reader, antenna, cable, warning, and action questions.
- Integration with antenna/cable degradation scoring.
- Local troubleshooting knowledge base.
- Responses through user app events or local web UI.
- Heartbeat monitoring for app CPU/RAM/status.

Future Enhancements

- Tiny local model after hardware benchmark.
- Local LAN LLM gateway for richer answers without cloud.
- Voice input from a local maintenance tablet.
- Automatic service-ticket draft on local network.
- Technician feedback loop for answer quality.
- Expanded knowledge base for all FXR90 operations.

Phase	Deliverable	Success Metric
Week 1-2	Skeleton user app, service startup, app install/start/autostart validation.	App runs reliably after reboot.
Week 3-4	Local telemetry cache, antenna/reader health lookup, first answer templates.	Answers top 10 maintenance questions.
Week 5-6	Local UI or request/event integration, SOP retrieval, technician-ready responses.	Technician can diagnose pilot issues without cloud.
After MVP	Resource hardening, auth, feedback capture, local gateway or tiny model evaluation.	Stable resource usage and useful answer quality.

OPERATIONAL SAFETY

Keep the chatbot helpful without risking reader stability

Because the chatbot runs on the reader, it must be resource-aware. It should never interfere with RFID inventory, reader gateway delivery, or critical device operations.

Risk	Impact	Control
High CPU/RAM	Could affect reader performance or user app stability.	Use lightweight rules first, set process limits, monitor `heartbeat.userapps[]` .
Large Model Size	Consumes storage and may slow startup.	Avoid heavy LLM in MVP; test tiny models only in lab.
Bad Advice	Technician may inspect wrong component or change wrong setting.	Use evidence-based templates, show confidence, avoid unsupported automatic changes.
Security	Unauthorized users may query device details.	Require local auth, restrict network binding, log access, and avoid exposing secrets.
Data Retention	Local storage can grow over time.	Rotate SQLite data, keep summaries, and enforce storage limits.

Resource KPIs

- Chatbot CPU and RAM usage.
- Startup time after reboot.
- Average answer latency.
- Local database size.
- App uptime and crash count.
- Reader inventory performance impact.

Answer Quality KPIs

- Top-question answer coverage.
- Technician usefulness rating.
- Correct root-cause suggestion rate.
- False or unsupported answer count.
- Time saved per maintenance incident.
- Cloud data transfer: zero for MVP.

BUSINESS VALUE

A private, practical AI assistant for field maintenance

This project gives the company an edge-AI story that is realistic: it keeps data local, uses the FXR90 user-app framework, and delivers immediate operational value by explaining device health and maintenance actions directly at the reader.

No Cloud Dependency

Maintenance answers remain available even without cloud connectivity.

Faster Troubleshooting

Technicians get the likely cause and next action at the edge.

Privacy Friendly

Reader telemetry and operational details stay local.

Best First Use Case

Pair the chatbot with the Antenna and Cable Degradation Predictor. This gives the assistant a clear job: explain weak antennas, cable risk, RF quality trends, and field-service steps from local evidence.

Stakeholder Pitch

We can deploy a no-cloud maintenance chatbot directly on the FXR90 as a user app. It will answer technician questions using local telemetry, explain AI health scores, and recommend safe maintenance actions without sending reader data outside the device or local network.

Recommended Next Step

Build a proof of concept that answers 10 predefined maintenance questions from local antenna/cable health data. Package it as a user app, run it on one test reader, and measure CPU, RAM, latency, answer quality, and operational impact.

Decision Point	Recommendation
First assistant style	Rules plus local retrieval, not a heavy on-device LLM.
First interface	Support `set_req_usr_app` / `userapp_event` plus optional local web UI.
First knowledge domain	Antenna/cable degradation and reader health troubleshooting.
Expansion	Evaluate a local LAN-based LLM gateway if richer natural-language behavior is required.

Prepared as a project concept based on the FXR90 MQTT API documentation in this workspace.